PROJECT III · REPORT 1

PROJECT III / TOOLS FOR PROGRAM DEVELOPMENT

Report 1 — Project Proposal

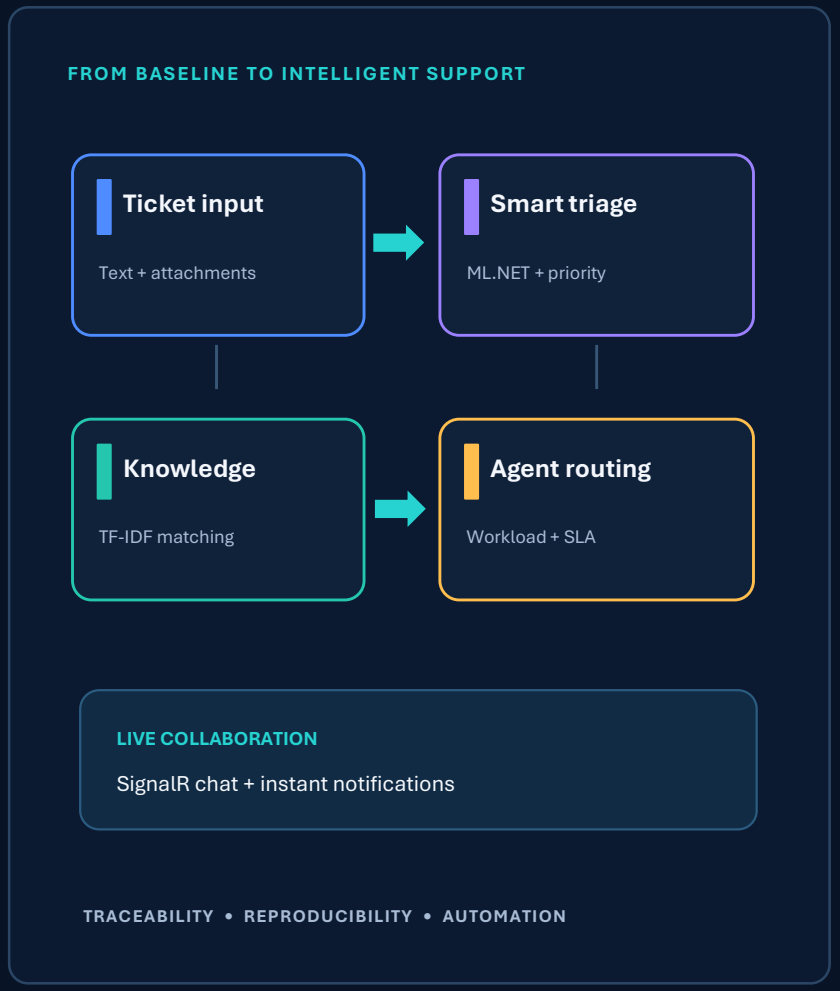
Intelligent IT Helpdesk Platform with Automated Classification & Incident Routing

Student

Șerban Alexandru-Cristian-George

Computer Engineering (English) · 4th Year

A functional helpdesk baseline, extended with intelligent triage, recommendations, balancing, and live updates.



01 / MOTIVATION

Why make the helpdesk intelligent?

The existing platform supports core ticketing; Project III targets the expensive manual steps around triage and resolution.

CURRENT PAIN POINTS

- Users often select the wrong category, department, or priority.
- Agents spend time re-routing tickets and diagnosing recurring issues.
- Ticket comments require manual refresh to see new replies.
- Unassigned tickets and approaching SLA deadlines can go unnoticed.

PROJECT OBJECTIVE

Turn the existing role-based web app into an intelligent support workflow.

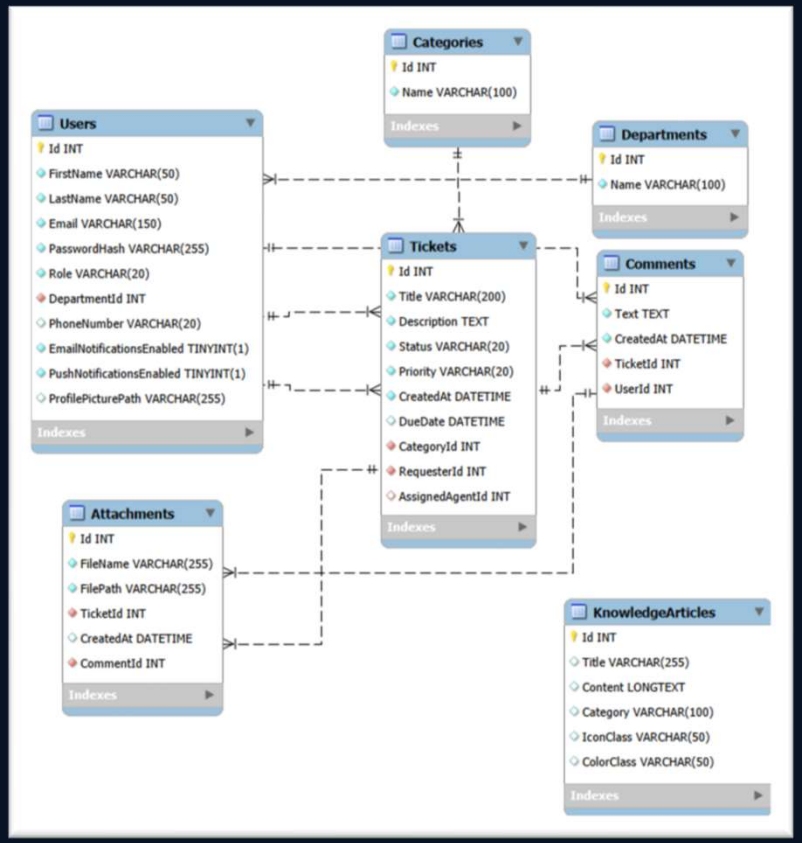
01	**Classify**	Suggest category, department, and priority
02	**Recommend**	Surface relevant articles and similar cases
03	**Route**	Balance workload and monitor SLA
04	**Connect**	Live chat and instant status alerts

EXPECTED DIRECTION

Less manual triage · **Faster reuse of knowledge** · **More balanced queues** · **Live communication**

Current state: a working N-tier baseline

The proposal extends the WAD Milestone 4 application built with ASP.NET Core MVC and Entity Framework Core.



EXISTING ARCHITECTURAL LAYERS

Presentation layer

MVC controllers, Razor views, role-specific layouts

Business logic / services

AuthService, TicketService, UserService, and other DI services

Data access layer

EF Core, HelpdeskContext, repositories, Unit of Work

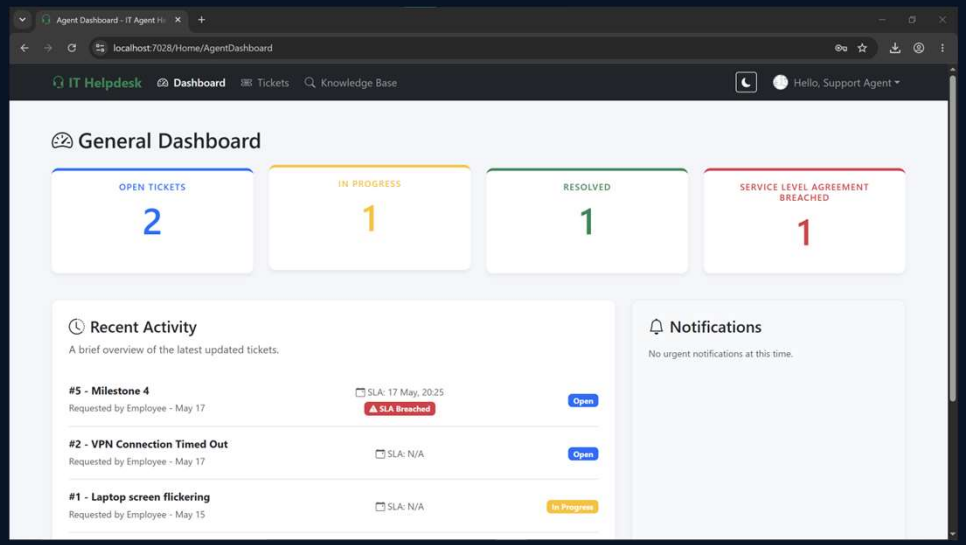
Persistent storage

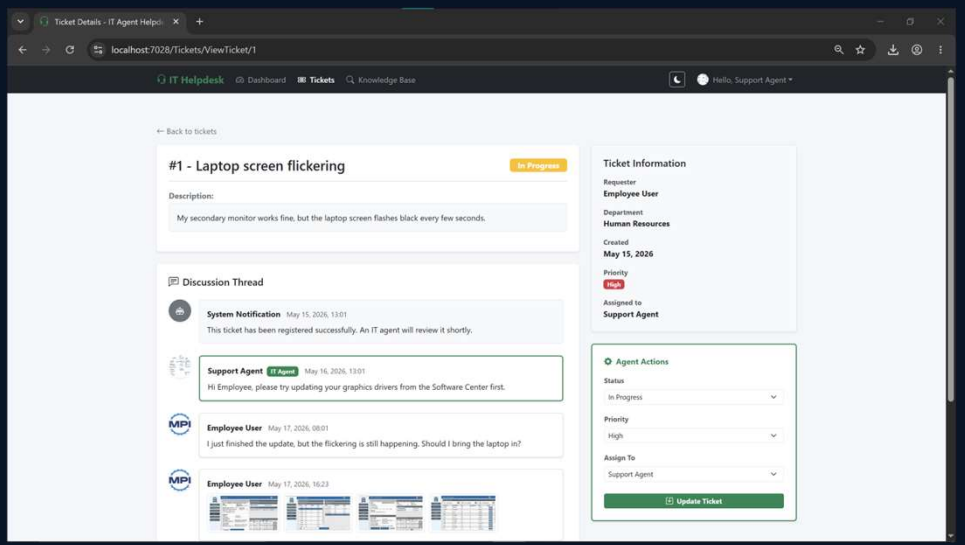
Relational data and seeded baseline records

Design principle: keep new modules inside the existing Service Layer and Dependency Injection pattern.

Existing platform: what already works

The project builds on role-based dashboards, ticket submission and tracking, comments, attachments, and a managed Knowledge Base.





Role-based access

Administrator · Support Agent · User · Guest

Ticket lifecycle

Submit, attach files, track status, and comment

Knowledge Base

Create, search, and browse help articles

Administration

Manage users, departments, and categories

04 / PROJECT SCOPE

Four enhancements planned for Project III

The scope evolves the existing ticket flow with automated classification, semantic recommendations, workload balancing, and real-time communication.

3.1 Automated classification

ML.NET

Predict Category, Department, and Priority from ticket title and description. Integrate with SubmitTicket.cshtml.

OUTPUT · structured ticket metadata

3.2 Semantic recommendations

TF-IDF + COSINE SIMILARITY

Recommend the top 3 Knowledge Base articles to users and similar resolved tickets to agents.

OUTPUT · relevant guidance

3.3 Workload balancing + SLA

TICKETSERVICE + IHOSTEDSERVICE

Assign tickets to the available agent with the lowest weighted workload; scan deadlines and escalate when needed.

OUTPUT · balanced queues

3.4 Real-time communication

ASP.NET CORE SIGNALR

Turn static comments into live user-agent conversation and show instant assignment, update, and escalation alerts.

OUTPUT · immediate updates

05 / PLANNED MODULE 1

Automated ticket classification & prioritization

ML.NET will assist users by predicting key ticket fields from the issue text at submission time.

TARGET: UNDER 100 MS PER REQUEST



IMPLEMENTATION NOTES

- Train on historical ticket titles and descriptions.
- Let the user leave category / priority blank for automatic prediction.

EXPECTED BENEFIT

More consistent triage

Predictions are a starting point for support workflow; the system still needs validation against representative historical data.

06 / PLANNED MODULE 2

Semantic recommendations: find known answers faster

TF-IDF vectors and cosine similarity will connect a new issue to existing knowledge and resolved tickets.

SELF-SERVICE + AGENT ASSISTANCE



USER SELF-SERVICE

SubmitTicket.cshtml

- As the description changes, call the service asynchronously via AJAX.
- Display the three best-matching Knowledge Base articles before ticket creation.

AGENT RESOLUTION ASSISTANT

ViewTicket.cshtml

- Compare the open issue with historical resolved tickets.
- Offer similar cases as diagnostic hints; agents retain decision authority.

Similarity quality depends on text preprocessing and the content of existing articles / tickets.

07 / PLANNED MODULE 3

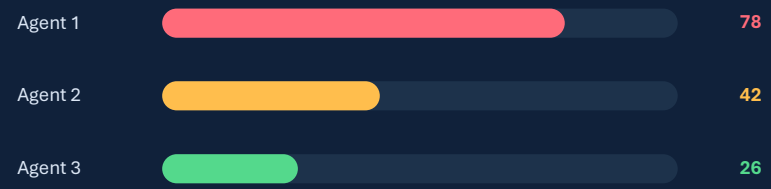
Smart agent workload balancing & SLA monitoring

Automated assignment reduces idle queues; a background worker helps catch response deadlines before they are missed.

WEIGHTED LEAST-LOADED ASSIGNMENT

New ticket → **TicketService**
Department + priority weighting

Illustrative active workload scores



Assign to Agent 3 — Lowest current workload score

SLA BACKGROUND WORKER

01 **Scan open tickets**
Periodic IHostedService execution

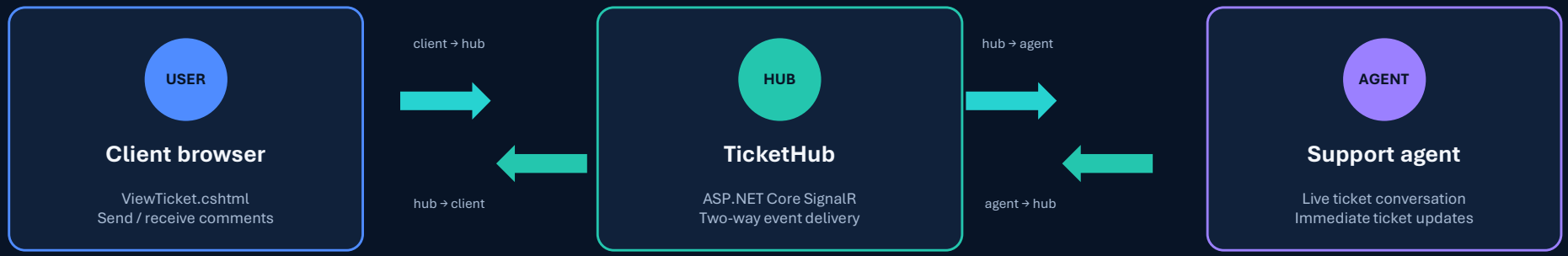
02 **Check SLA deadlines**
Compare elapsed time with target response window

03 **Escalate when needed**
Raise priority as the deadline approaches or is breached

Target: proactive escalation + clearer ownership

Real-time communication with SignalR

Replace the current refresh-to-see-comments experience with a live conversation and immediate assignment / update notifications.



REAL-TIME EVENTS

Ticket assigned · New comment · Status updated · SLA escalated

Performance target from the proposal: SignalR notifications under 50 ms on the local network.

Implementation touchpoints: TicketHub · _AgentLayout.cshtml · _UserLayout.cshtml

09 / SYSTEM SPECIFICATIONS

Actors, functional requirements & quality targets

The new modules fit into the existing role-based access model and must preserve maintainability and traceability.

G Guest

EXISTING — Browse public Knowledge Base • Authenticate

PLANNED — No new privileged actions

U User / Client

EXISTING — Submit tickets + attachments • Track tickets and comment

PLANNED — Get article suggestions • Use live chat / instant alerts

SA Support Agent

EXISTING — View queues and statistics • Publish troubleshooting articles

PLANNED — See similar resolved tickets • Receive balanced assignments

A Administrator

EXISTING — Manage users, roles, teams • Manage departments/categories

PLANNED — Monitor SLA breach metrics • Trigger model retraining

NON-FUNCTIONAL TARGETS

- **ML / TF-IDF < 100 ms**
- **SignalR < 50 ms on LAN**
- **Service-layer modularity**
- **GitHub traceability**

10 / DEVELOPMENT TOOLCHAIN

Technology stack: existing foundations + planned modules

New capabilities are designed to fit the project's current C# / ASP.NET Core architecture.

IDE & DEBUGGING

Visual Studio 2022 / VS Code

C# compilation, Razor editing, debugger, CPU / memory diagnostics

WEB FRAMEWORK

ASP.NET Core MVC + EF Core

Controllers, views, dependency injection, LINQ, ORM

TICKET ML

ML.NET

Multiclass text classification with SdcaMaximumEntropy

RECOMMENDATION ENGINE

TF-IDF + cosine similarity

Custom C# text vectors and ranked recommendations

LIVE EVENTS & JOBS

SignalR + IHostedService

Two-way chat, toast alerts, and SLA deadline scanning

FRONTEND

Bootstrap 5 · Razor · jQuery

Responsive layouts, validation, and LibMan client libraries

DATABASE & MODELING

SQL Server / SQLite · MySQL Workbench

Relational storage and EER/schema modeling

TRACEABILITY & TESTING

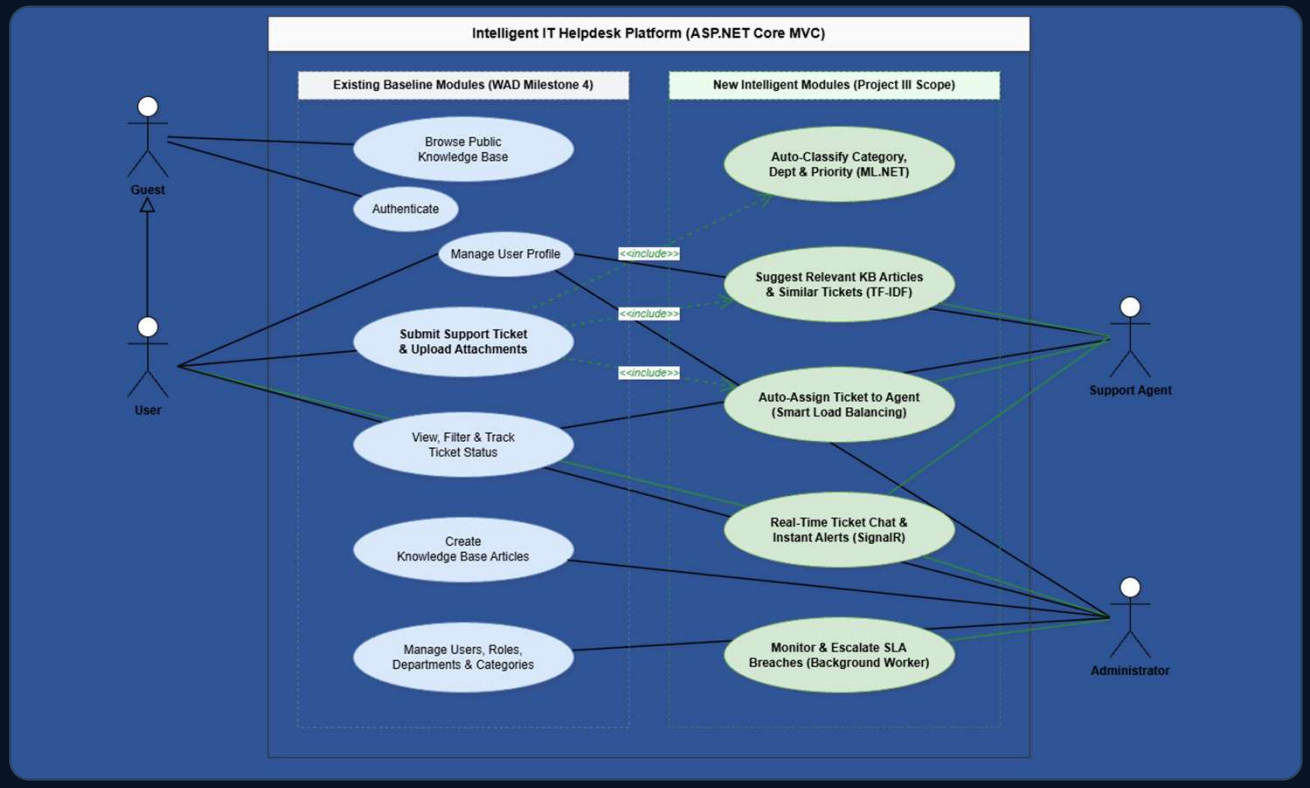
Git · GitHub · Draw.io · Postman

Issues, milestones, commits, use-case modeling, API checks

Baseline rule: new modules should use the existing Service Layer + Dependency Injection pattern.

Use Case Diagram: current baseline vs. new modules

The diagram distinguishes existing WAD functionality from planned Project III capabilities.



LEGEND

Existing baseline
WAD Milestone 4

New Project III
Planned intelligent modules

Actors

- Guest
- User / Client
- Support Agent
- Administrator

Ticket submission automatically triggers ML classification, TF-IDF matching, and smart routing via <<include>> relationships.

11-week roadmap with visible project traceability

Milestones align with the course plan: proposal → minimal prototype → complete implementation and demonstration.

WEEKS 1–3

Report 1

Project Proposal

Specifications, selected tools, baseline architecture, and Use Case Diagram.

CURRENT STAGE

WEEKS 4–7

Report 2

Minimal Design & Prototype

Prototype TF-IDF recommendations, ML.NET classification, and smart assignment.

PROTOTYPE

WEEKS 8–11

Final Report

Implementation & Demo

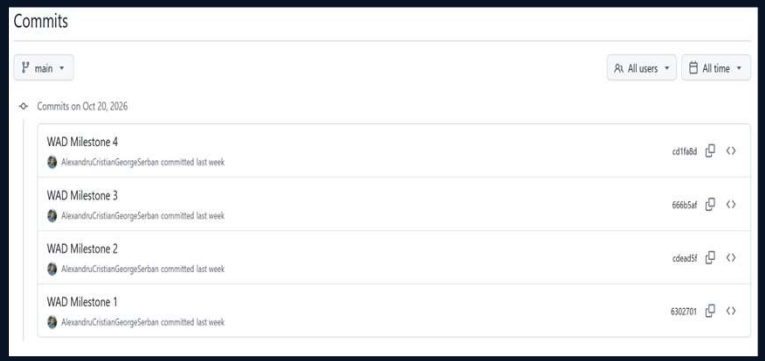
SignalR chat, SLA worker, end-to-end testing, metrics, and complete demo.

DELIVERY

TRACEABILITY WORKFLOW

GitHub Issue → Atomic commit → Milestone → Report / Demo

Track features, bug fixes, and milestones; use commits that correspond to one focused change.



A practical path from ticketing to intelligent support

The proposal adds intelligence around an existing application while preserving its layered architecture and role model.

01 Reuse the baseline

Extend the existing ASP.NET Core MVC application rather than rebuilding its core ticketing workflow.

02 Automate triage

Use ML.NET and semantic similarity to help route tickets and surface already-known solutions.

03 Improve operations

Balance assignments, monitor SLA deadlines, and notify users and agents in real time.

04 Make progress verifiable

Track implementation with Issues, milestones, atomic commits, prototype evidence, and a final demo.

NEXT STEP **Finalize Report 1 artifacts, commit them to GitHub, and begin the Report 2 prototype.**